

TECH MAHINDRA

■4.5 LPA Test Preparation Guide

7-Day Intensive Crash Course

Goal: Prepare for the package-upgradation assessment and maximize the chance of qualifying for the higher package.

Includes: Quantitative Aptitude • Logical Reasoning • English • Programming • DSA • OOP • SQL • DBMS • Operating Systems • Computer Networks • Coding Practice • Mock-Test Strategy

Important: The exact campus test pattern can vary. Treat the instructions received from your university/Tech Mahindra as the final authority.

How to Use This Guide

This guide expands every topic from the 7-day crash plan into a compact but detailed revision handbook. The objective is not to memorize everything once; it is to understand the pattern, solve examples, identify mistakes, and then repeat timed practice.

Recommended daily method: Learn the concept → study the worked example → solve questions without looking → check mistakes → revise the same topic at night.

Day	Main Focus	Target
1	Quantitative aptitude + programming basics	50 aptitude + 7 coding problems
2	Logical reasoning + OOP	50–60 reasoning + OOP revision
3	SQL + DBMS	30–50 SQL/DBMS questions
4	DSA + coding	10–15 coding problems
5	CN + OS + English	Technical revision + communication practice
6	Full mock	1 complete timed mock + deep analysis
7	Final revision	2nd mock + mistake revision

Exam Tip: Do not try to finish the PDF passively. Your score improves from solving questions under time pressure, not from reading alone.

1. Quantitative Aptitude

Quantitative aptitude tests your ability to translate a word problem into arithmetic or algebra quickly. In placement tests, speed matters almost as much as correctness.

1.1 Percentages

Percentage means “per hundred.” If a quantity changes from A to B, percentage change is: $(B - A) / A \times 100$.

Example — Increase

A salary rises from ₹20,000 to ₹24,000. Increase = ₹4,000. Percentage increase = $4,000 / 20,000 \times 100 = 20\%$.

Example — Decrease

A price falls from ₹800 to ₹680. Decrease = ₹120. Percentage decrease = $120 / 800 \times 100 = 15\%$.

Successive percentage changes

For increases/decreases of a% and b%, the net multiplier is $(1 \pm a/100)(1 \pm b/100)$.

Example: +20% followed by –10% → $1.20 \times 0.90 = 1.08$, so the net increase is **8%**, not 10%.

Exam Tip: When a question says “increased by x% and then decreased by x%,” the final value is lower than the original: net change = $-x^2/100$ percent.

1.2 Ratio and Proportion

A ratio compares quantities. 2:3 means that for every 2 units of the first quantity there are 3 of the second.

If A:B = 2:3 and A+B = 50, total parts = 5. One part = 10, so A = 20 and B = 30.

Proportion

If $a/b = c/d$, then $a \times d = b \times c$. This cross-multiplication is useful for direct calculations.

Example

5 workers complete a task in 12 days. Assuming equal efficiency, 10 workers take 6 days. This is an inverse relationship: workers × days remains constant.

1.3 Averages

Average = Sum of observations / Number of observations.

Example: Average of 10, 15, 20, 25 = $70/4 = 17.5$.

Replacement shortcut

If one value x is replaced by y in a group of n items, change in average = $(y-x)/n$.

Example: In 20 students, a score 45 is replaced by 65. Average increases by $20/20 = 1$.

1.4 Profit, Loss and Discount

Profit = Selling Price – Cost Price. Loss = Cost Price – Selling Price.

Profit% = Profit/CP $\times$ 100. Loss% = Loss/CP $\times$ 100.

Discount = Marked Price – Selling Price. Discount% = Discount/MP $\times$ 100.

Example

CP = ₹500, SP = ₹575. Profit = ₹75. Profit% = $75/500 \times 100 = 15\%$.

Successive discounts

A 20% discount followed by 10% discount is not 30%. Price multiplier = $0.8 \times 0.9 = 0.72$, so total discount = **28%**.

1.5 Simple Interest and Compound Interest

Simple Interest: **SI = PRT/100**, where P = principal, R = annual rate, T = time in years.

Compound amount: **$A = P(1 + R/100)^T$** ; CI = A – P.

Example

₹10,000 at 10% simple interest for 2 years: SI = $10,000 \times 10 \times 2/100 = ₹2,000$.

For annual compounding over 2 years: $A = 10,000 \times 1.1^2 = ₹12,100$, so CI = **₹2,100**.

1.6 Number System

Know divisibility rules, factors, multiples, primes, LCM, HCF/GCD, remainders, odd/even properties, and basic modular arithmetic.

Rule	Shortcut
2	Last digit even
3	Sum of digits divisible by 3
5	Last digit 0 or 5
9	Sum of digits divisible by 9
10	Last digit 0
11	Alternating digit sum is divisible by 11

HCF and LCM

For two positive integers a and b : $a \times b = \text{HCF}(a,b) \times \text{LCM}(a,b)$.

Example: 12 and 18 $\rightarrow$ HCF = 6, LCM = 36, and $12 \times 18 = 6 \times 36 = 216$.

1.7 Time and Work

If A completes a job in x days, A's one-day work = $1/x$. Combined work rates are added.

Example: A takes 10 days and B takes 15 days. Together: $1/10 + 1/15 = 1/6$, so they take **6 days**.

Exam Tip: Think in work-rates, not days. Convert every worker into “work per day.”

1.8 Time, Speed and Distance

Speed = Distance/Time. Therefore Distance = Speed×Time and Time = Distance/Speed.

Unit conversion: 1 m/s = 18/5 km/h; 1 km/h = 5/18 m/s.

Example

A car travels 180 km in 3 hours. Speed = $180/3 = 60$ km/h.

Relative speed

For objects moving in opposite directions, relative speed is the sum. In the same direction, it is the difference.

1.9 Probability

For equally likely outcomes: Probability = Favorable outcomes / Total outcomes.

Example: Probability of getting an even number on a fair die = $3/6 = 1/2$.

Complement: $P(\text{not } A) = 1 - P(A)$. This is often the fastest route.

1.10 Permutation and Combination

Permutation is arrangement where order matters. $nPr = n!/(n-r)!$. Combination is selection where order does not matter. $nCr = n!/[r!(n-r)!]$.

Example: Number of ways to arrange 3 people from 5 = $5P3 = 5 \times 4 \times 3 = 60$.

Number of ways to select 3 people from 5 = $5C3 = 10$.

2. Logical Reasoning

2.1 Number and Alphabet Series

Look for differences, ratios, alternating patterns, squares/cubes, prime numbers, or separate odd/even positions.

Example: 2, 5, 10, 17, 26, ? Differences are 3, 5, 7, 9, so next difference = 11 → **37**.

Alphabet series can use positions A=1, B=2, ... Z=26.

2.2 Coding and Decoding

A word may be transformed using letter shifts, reversal, position values, or a substitution pattern.

Example: If CAT becomes DBU by shifting every letter +1, DOG becomes **EPH**.

Exam Tip: Write the alphabet positions when the pattern is not obvious.

2.3 Blood Relations

Translate each relationship into a small family tree. Identify gender only when explicitly stated or logically fixed.

Example: "A is the brother of B. B is the mother of C." A is C's **maternal uncle**.

2.4 Direction Sense

Use North, South, East and West on a coordinate grid. Treat each move as a vector and combine horizontal and vertical displacement.

Example: 5 km north, then 3 km east. Final displacement = $\sqrt{(5^2+3^2)} = \sqrt{34} \approx 5.83$ km.

2.5 Odd One Out and Analogies

Identify the shared property among most items. For analogies, determine the relationship between the first pair and apply the same relationship to the second pair.

Example: 2, 3, 5, 9 → 9 is odd because the others are prime.

2.6 Syllogisms

Syllogism questions give statements and ask which conclusions must follow. Use Venn diagrams or set logic.

Example: All cats are animals. Some animals are black. You cannot conclude that some cats are black. The second statement does not guarantee overlap with cats.

Exam Tip: Choose only conclusions that are definitely true, not merely possible.

2.7 Seating Arrangements

For linear arrangements, draw slots. For circular arrangements, fix one person to remove rotational duplicates. Track left/right carefully.

Example: If A is immediately left of B, write [A][B] as a fixed block before placing other people.

2.8 Data Interpretation

Read tables, bar charts, pie charts and line graphs. Convert percentages into actual values and compare ratios.

If total sales are 500 and one category is 24%, that category = $500 \times 0.24 = 120$.

3. Programming Fundamentals

3.1 Variables and Data Types

A variable stores a value. Common types include integer, floating-point, character, boolean and string. Exact syntax varies by language.

Example in C++

```
int age = 21;
double salary = 45000.50;
char grade = 'A';
bool selected = true;
string name = "Aditya";
```

Know type ranges conceptually and understand integer division. For example, $5/2$ is 2 in integer arithmetic in C++.

3.2 Loops

Use for loops when the number of repetitions is known; while loops when repetition depends on a condition.

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
```

Output: 1 2 3 4 5.

3.3 Functions

A function packages reusable logic. Know parameters, return values, scope, and pass-by-value vs reference.

```
int add(int a, int b) {
    return a + b;
}
```

3.4 Arrays

An array stores elements of the same type in contiguous memory in languages such as C/C++. Indexing commonly starts at 0.

```
int a[] = {10, 20, 30, 40};
cout << a[2]; // 30
```

Basic traversal is $O(n)$. Direct indexed access is typically $O(1)$.

3.5 Strings

Know length, indexing, comparison, concatenation, reversal, character frequency and palindrome checks.

```
string s = "level";
string r = s;
reverse(r.begin(), r.end());
bool palindrome = (s == r);
```

3.6 Recursion

Recursion means a function calls itself. Every recursive solution needs a base case and a smaller subproblem.

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Without a valid base case, recursion can continue until stack overflow.

3.7 Time Complexity

Big-O describes how runtime grows with input size.

Complexity	Typical example
O(1)	Array index access
O(log n)	Binary search
O(n)	Linear scan
O(n log n)	Efficient comparison sorting
O(n ²)	Nested loops over n items

Exam Tip: For nested loops, ask how many times the inner loop executes in total. Not every nested loop is automatically O(n²).

4. Object-Oriented Programming

4.1 Class and Object

A class is a blueprint; an object is an instance of that class.

```
class Car {
public:
    string model;
    void start() {
        cout << "Engine started";
    }
};

Car c1;
c1.model = "Honda";
c1.start();
```

4.2 Encapsulation

Encapsulation combines data and methods and restricts direct access to internal state. A common technique is private data with public getters/setters.

Example: A bank account can keep balance private and expose deposit/withdraw methods that enforce rules.

4.3 Abstraction

Abstraction exposes what an object can do while hiding unnecessary implementation details.

Example: You call car.start(); you do not need to know every internal engine step.

4.4 Inheritance

Inheritance allows a derived class to reuse or extend a base class.

```
class Animal {
```

```

public:
    void eat() { cout << "Eating"; }
};

class Dog : public Animal {
public:
    void bark() { cout << "Barking"; }
};

```

Dog can use inherited eat() and its own bark().

4.5 Polymorphism

Polymorphism means one interface can represent different behaviors. Common forms are compile-time overloading and runtime overriding.

Overloading: same function name, different parameter lists. Overriding: derived class provides a different implementation of a base-class method.

4.6 Constructor

A constructor initializes an object and is automatically called when an object is created.

```

class Student {
public:
    Student(string n) {
        name = n;
    }
private:
    string name;
};

```

4.7 Overloading vs Overriding

Overloading	Overriding
Usually same class	Base/derived classes
Different parameter list	Same method signature conceptually
Compile-time polymorphism	Runtime polymorphism when virtual dispatch is used

4.8 Access Modifiers

public: accessible from outside. private: accessible only inside the class. protected: accessible in the class and derived classes, subject to language rules.

5. SQL

5.1 SELECT, WHERE, ORDER BY

```

SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;

```

SELECT chooses columns. FROM chooses the table. WHERE filters rows. ORDER BY sorts the result.

5.2 DISTINCT, LIKE, IN, BETWEEN

```

SELECT DISTINCT department FROM employees;

```

```

SELECT * FROM employees
WHERE name LIKE 'A%';

```

```

SELECT * FROM employees
WHERE department IN ('IT', 'HR');

```

```

SELECT * FROM employees
WHERE salary BETWEEN 30000 AND 60000;

```

LIKE 'A%' means starts with A. BETWEEN is commonly inclusive at both ends in SQL.

5.3 Aggregate Functions

COUNT, SUM, AVG, MIN and MAX summarize data.

```
SELECT department, COUNT(*) AS employee_count,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department;
```

5.4 GROUP BY and HAVING

GROUP BY creates groups. HAVING filters groups after aggregation. WHERE filters rows before grouping.

```
SELECT department, COUNT(*) AS cnt  
FROM employees  
WHERE salary > 30000  
GROUP BY department  
HAVING COUNT(*) >= 5;
```

5.5 Joins

INNER JOIN returns matching rows. LEFT JOIN keeps all rows from the left table and matching rows from the right table.

```
SELECT e.name, d.department_name  
FROM employees e  
INNER JOIN departments d  
ON e.department_id = d.id;
```

Join	Idea
INNER	Only matching rows
LEFT	All left rows + matching right rows
RIGHT	All right rows + matching left rows
SELF	Table joined with itself

5.6 Second Highest Salary

```
SELECT MAX(salary) AS second_highest  
FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees);
```

This is a classic interview question. Be aware that duplicate salaries can require a DISTINCT-based or ranking solution depending on the requirement.

5.7 Duplicate Records

```
SELECT email, COUNT(*) AS cnt  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

6. DBMS

6.1 Primary Key

A primary key uniquely identifies each row and cannot contain NULL values.

6.2 Foreign Key

A foreign key references a key in another table and helps maintain referential integrity.

Example: employees.department_id can reference departments.id.

6.3 Candidate Key

A candidate key is a minimal set of attributes that can uniquely identify a record. One candidate key is chosen as the primary key.

6.4 Normalization

Normalization organizes data to reduce redundancy and update anomalies.

Normal Form	Core idea
1NF	Atomic values; no repeating groups
2NF	1NF + no partial dependency on part of a composite key
3NF	2NF + no transitive dependency of non-key attributes

Example: If student_id determines department_id and department_id determines department_name, storing department_name repeatedly in every student row creates transitive dependency. Separate department data into its own table.

6.5 ACID Properties

Atomicity: transaction is all-or-nothing. **Consistency:** rules/constraints remain valid. **Isolation:** concurrent transactions should not improperly interfere. **Durability:** committed changes survive failures.

6.6 Transactions

A transaction is a logical unit of work. Common commands include COMMIT and ROLLBACK. SAVEPOINT can mark a point to which a transaction can roll back.

6.7 Indexing

An index speeds up many read/search operations by providing a faster lookup structure, but indexes consume storage and can slow inserts/updates because the index must also be maintained.

Exam Tip: Interview answer: "Indexes improve query performance for suitable lookups, but too many indexes increase write cost and storage."

6.8 DELETE vs TRUNCATE vs DROP

Command	Effect
DELETE	Removes selected rows; WHERE can be used
TRUNCATE	Removes all rows quickly; table structure remains
DROP	Removes the table/object itself

7. Data Structures and Algorithms

7.1 Linear Search

Check each element until the target is found. Time complexity is $O(n)$.

```
for (int i = 0; i < n; i++) {
    if (a[i] == target) return i;
}
return -1;
```

7.2 Binary Search

Binary search works on a sorted array. Compare the target with the middle element and discard half the search space each time. Complexity is $O(\log n)$.

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == target) return mid;
```

```

    if (a[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;

```

7.3 Bubble, Selection and Insertion Sort

Bubble sort repeatedly swaps adjacent out-of-order values. Selection sort repeatedly selects the minimum remaining value. Insertion sort inserts each value into the sorted portion.

Algorithm	Typical time	Idea
Bubble	$O(n^2)$	Adjacent swaps
Selection	$O(n^2)$	Select minimum
Insertion	$O(n^2)$	Insert into sorted prefix

7.4 Two Sum

Goal: find two values that add to a target. A hash map can store values already seen, giving average $O(n)$ time.

```

unordered_map<int,int> pos;
for (int i = 0; i < n; i++) {
    int need = target - a[i];
    if (pos.count(need)) {
        // pair found
    }
    pos[a[i]] = i;
}

```

7.5 Maximum Subarray — Kadane's Algorithm

Maintain the best sum ending at the current position. At each value, choose between starting a new subarray or extending the current one. Complexity $O(n)$.

```

long long cur = a[0], best = a[0];
for (int i = 1; i < n; i++) {
    cur = max((long long)a[i], cur + a[i]);
    best = max(best, cur);
}

```

7.6 Common Coding Problems

Problem	Core concept
Reverse string	Two pointers / library reverse
Palindrome	Compare mirrored characters
Anagram	Frequency counts / sorting
Frequency	Hash map / array counts
Second largest	Track top two values
Missing number	Sum/XOR techniques
Move zeros	Two-pointer compaction
Rotate array	Index mapping / reversal
Binary search	Sorted search space

8. Operating Systems

8.1 Process vs Thread

A process is an executing program with its own address space/resources. A thread is a smaller execution unit within a process and usually shares the process's memory/resources.

Process	Thread
More isolated	Shares process resources
Higher creation/context cost generally	Usually lighter
Separate address space	Same process address space

8.2 Deadlock

Deadlock occurs when processes wait indefinitely for resources held by one another.

The four classic necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait.

Exam Tip: Remember the acronym-like chain: exclusive resource + holding while waiting + cannot forcibly take + circular dependency.

8.3 Scheduling

Know FCFS, SJF, Round Robin and Priority scheduling conceptually. Round Robin uses a time quantum and is common in interactive systems.

Be able to calculate waiting time and turnaround time for simple scheduling tables.

8.4 RAM vs ROM

RAM is volatile working memory. ROM is non-volatile memory used for firmware or persistent instructions. Modern systems also use flash/EEPROM and other non-volatile technologies.

8.5 Virtual Memory and Paging

Virtual memory lets a system use disk/SSD as an extension of memory when needed. Paging divides virtual/physical memory into fixed-size pages/frames. A page fault occurs when a required page is not currently in physical memory.

8.6 Context Switching

The OS saves the state of one process/thread and restores another so the CPU can switch execution. Context switching has overhead.

9. Computer Networks

9.1 OSI Model

Layer	Name	Typical examples
7	Application	HTTP, DNS, SMTP
6	Presentation	Encoding/encryption concepts
5	Session	Session management
4	Transport	TCP, UDP
3	Network	IP, routing
2	Data Link	Ethernet, MAC
1	Physical	Signals, cables, radio

Mnemonic: “Please Do Not Throw Sausage Pizza Away” is one common way to remember layers from Physical upward, but any mnemonic is fine.

9.2 TCP vs UDP

TCP is connection-oriented and provides reliable, ordered delivery with retransmission mechanisms. UDP is connectionless and has lower overhead but does not guarantee delivery or ordering.

TCP	UDP
Reliable/ordered	No built-in delivery guarantee
Connection-oriented	Connectionless
More overhead	Lower overhead
Common for web/data transfer	Common for real-time/latency-sensitive traffic

9.3 HTTP vs HTTPS

HTTPS is HTTP protected by TLS. It provides encryption in transit and server authentication through certificates, helping prevent eavesdropping and tampering.

9.4 GET vs POST

GET generally retrieves data and is intended to be safe/idempotent in normal REST use. POST generally submits data or requests creation/action and can change server state.

Do not assume “GET is always secure” or “POST encrypts data.” Security comes from HTTPS and correct application design.

9.5 IP Address

An IP address identifies a network interface/address in an IP network. IPv4 uses 32 bits; IPv6 uses 128 bits.

9.6 DNS

DNS translates domain names such as example.com into IP addresses or other records so clients can locate services.

9.7 MAC Address

A MAC address is a link-layer hardware/network-interface identifier used in local network communication. It is distinct from an IP address.

9.8 Router vs Switch

A switch primarily connects devices within a LAN and forwards frames based on MAC addresses. A router connects networks and forwards packets based on IP routing.

10. English and Communication

10.1 Reading Comprehension

Read the question first when the passage is long. Identify the author's main idea, tone, evidence, and exact wording. Avoid selecting an option merely because it uses words from the passage.

10.2 Grammar

Revise subject-verb agreement, tenses, articles, prepositions, pronouns, conjunctions and modifiers.

Example: “Each of the students **has** submitted the form,” not “have,” because the subject is “each.”

10.3 Sentence Correction

Look for tense mismatch, subject-verb mismatch, incorrect preposition, parallelism errors and unnecessary words.

Incorrect: “He is good in coding.” Better: “He is good **at** coding.”

10.4 Vocabulary

Build vocabulary through context rather than memorizing huge word lists. Learn synonyms, antonyms, prefixes/suffixes and common business words.

Example: “mitigate” = reduce or make less severe.

10.5 Fill in the Blanks

Read the entire sentence, predict the missing idea, then compare options. Check grammar as well as meaning.

10.6 Para Jumbles

Find the opening sentence first. Look for pronouns that refer backward, connectors such as however/therefore, and chronological or logical progression.

10.7 Spoken Communication

For interviews, use a simple structure: answer directly → explain briefly → give an example. Avoid memorized paragraphs that sound unnatural.

Example: "I am comfortable with React and Node.js. I have used React for frontend development and Node/Express for APIs, including authentication and database integration."

11. Mock-Test and Exam Strategy

11.1 Before the Test

- Keep your laptop charged and test your browser/internet if the test is online.
- Keep required ID and college information available.
- Read all instructions before starting.
- Do not open prohibited tabs, tools or external resources.
- Sleep properly; speed and accuracy suffer when you are exhausted.

11.2 During the Test

Use a three-pass strategy when navigation is available:

- Pass 1: answer immediate/easy questions.
- Pass 2: solve medium questions.
- Pass 3: return to lengthy questions.

For coding, first understand input/output and constraints, then write a correct simple solution, then optimize if needed.

11.3 Mistake Analysis

Mistake type	Action
Concept unknown	Study concept + solve 5 similar questions
Calculation error	Slow down at final step; use estimation
Misread question	Underline key words mentally
Time pressure	Practice timed sets
Coding syntax error	Write/compile small programs regularly

12. Detailed 7-Day Schedule

Day	Morning	Afternoon	Evening
1	Percentages, ratio, average	Profit/loss, SI/CI, number system	Programming basics + 20 timed questions
2	Series, coding-decoding	Blood relation, directions, syllogism	OOP + 20 reasoning questions
3	SQL basics	Joins, GROUP BY, subqueries	DBMS + 30 SQL questions
4	Arrays, strings	Searching/sorting	Coding problems + complexity

Day	Morning	Afternoon	Evening
5	OS	Computer Networks	English + communication
6	Full mock	Analyze every mistake	Weak-topic revision
7	Formula/CS revision	Full mock #2	Final revision + light practice

Minimum Daily Targets

- Aptitude/reasoning: 40–60 questions
- Coding: 3–5 problems
- Technical MCQs: 25–40
- SQL: 5–10 queries
- English: 20–30 questions
- Revision: 30 minutes

13. Coding Practice Sheet

Solve these without looking at solutions. First write the approach in plain English, then code, then test edge cases.

- 1. Reverse a string.
- 2. Check whether a string is a palindrome.
- 3. Check whether two strings are anagrams.
- 4. Find the largest element in an array.
- 5. Find the second largest distinct element.
- 6. Count frequency of every element.
- 7. Move all zeroes to the end while preserving non-zero order.
- 8. Find the missing number from 1..n.
- 9. Solve Two Sum.
- 10. Perform binary search.
- 11. Reverse an array in-place.
- 12. Rotate an array by k positions.
- 13. Find maximum subarray sum.
- 14. Count vowels and consonants.
- 15. Find the first non-repeating character.

Edge Cases You Must Test

- Empty input
- One element
- All elements equal
- Already sorted input
- Reverse-sorted input
- Negative values
- Duplicate values
- Very large values

14. High-Frequency Technical Questions

Question	Strong short answer
What is OOP?	A programming paradigm centered around objects that combine data and behavior; key concepts include encapsulation, inheritance, and polymorphism.
What is the difference between SQL and NoSQL?	Databases commonly use structured relational schemas and SQL; NoSQL is a broad family including document, key-value, columnar, and graph databases.
What is a primary key?	A column or set of columns that uniquely identifies rows and cannot be NULL.
What is a foreign key?	A column/set of columns referencing a key in another table, used for relational integrity.
What is normalization?	Organizing relational data to reduce redundancy and anomalies.
What is an index?	A data structure that can accelerate suitable searches at the cost of storage and write overhead.
Process vs thread?	A process is an isolated execution environment; threads are execution units within a process and generally share resources.
TCP vs UDP?	TCP provides reliable ordered transport; UDP provides lower-overhead connectionless transport without delivery guarantees.
What is binary search?	A search algorithm for sorted data that halves the search space each step, $O(\log n)$.
What is Big-O?	A notation describing how resource usage grows asymptotically with input size.
What is a deadlock?	A state where processes/threads wait indefinitely for resources held by one another.
What is HTTPS?	HTTP transported over TLS, providing encryption in transit and server authentication.

15. Final 24-Hour Checklist

- Revise percentage, ratio, average, profit/loss and time-work formulas.
- Revise series, coding-decoding, syllogism and direction questions.
- Revise OOP four pillars and overloading vs overriding.
- Write at least 5 SQL queries from memory.
- Revise joins, GROUP BY/HAVING, keys and normalization.
- Solve 3–5 easy coding problems without assistance.
- Revise OSI model, TCP vs UDP, HTTP vs HTTPS and DNS.
- Revise process vs thread and deadlock.
- Take a timed mini-test.
- Review mistakes rather than learning a large new topic.
- Prepare your documents, laptop and internet if applicable.
- Sleep adequately and avoid an all-night study session.

Final Mindset

Target the higher package. Do not prepare only to qualify. Your strongest return on time will come from mastering recurring placement patterns, becoming fast at basic aptitude, and being able to write clean solutions for easy-to-medium coding problems.

The exact score/cutoff required for the ■4.5 LPA package is campus- and drive-specific unless Tech Mahindra/SPCR provides an official cutoff. Therefore, use this guide to maximize your score rather than assuming a particular cutoff.